Lesson notes · for you

Tell the computer

Optimization · Level 2 · Lesson 4

Time	Materials
20 min	A phone with Darsi (or a friend's), or paper and a pen; Your garden graph from the last lesson

Learning goals

- Read a small model in Python: variables, limits, a goal.
- Put a model's lines in order and fill its gaps.
- Predict what it prints, and run it in the lab.

Activity: Predict, then run

1. Read (5 min): Open the "Truck load" example in the modeller lab. Point to the yes-or-no variables, the weight limit and the goal, and say each in your own words.
2. Predict (5 min): Write what you think happens with capacity 1100, with capacity 800, and with a rule that the water filter must go.
3. Run (7 min): Make each change, run it, and write what the solver chose next to your prediction.
4. Share (3 min): Tell a partner which prediction was wrong and what you learned from it.

Python downloads the first time (about 0.3 MB): run one example while online. No phone? Do the activity on paper with the garden model.

Discussion

- Which line of the program is the goal, and how can you tell?
- When would you get INFEASIBLE?
- What rule would make a school plan fairer?

Check yourself

- Write one limit as a model.add(...) line and say what it means.

Try it this week

In the modeller lab, open the "Truck load" example. Before you run it, guess what changes when capacity is 1100, then run it and check. No phone? Write the garden model on paper, line by line, and say what each line means.

Worksheet

Tell the computer

Optimization · Level 2 · Lesson 4

Name _____ Class _____ Date _____

1 **You add a rule to the exam-slots model and run it. The solver prints INFEASIBLE. What does that mean?**

◯ No plan keeps all the rules ◯ The phone is too slow ◯ The program has a typo

2 **This year the garden has only bean beds: each takes 3 of your hours a week, and you have 18. Put the lines of this program in the order they run.**

Number the lines in order (1, 2, 3…). Cross out any line that doesn't belong.

☐ `model.add(3 * y <= 18)`

☐ `model.maximize(40 * y)`

☐ `from ortools.sat.python import cp_model`

☐ `model.add(3 * y >= 18)`

☐ `y = model.new_int_var(0, 10, "beans")`

☐ `model = cp_model.CpModel()`

☐ `print(solver.value(y))`

☐ `print(y)`

☐ `solver = cp_model.CpSolver()`

☐ `solver.solve(model)`

3 **What does this program print?**

```
x = 3
y = 5
print(x + y <= 8, 30 * x + 40 * y)
```

◯ True 290 ◯ False 290 ◯ 8 290

4 **Complete the garden model: a whole-number variable, the land limit (at most 8 beds) and the goal (the most profit).**

```
x = model. _____ (0, 10, "tomato")
y = model.new_int_var(0, 10, "beans")
model.add(x + y  _____  8)
model.add(2 * x + y <= 14)
model.add(x + 3 * y <= 18)
model. _____ (30 * x + 40 * y)
```

Word bank: `>=` `maximize` `<=` `print` `new_int_var` `minimize`

5 **The same program runs unchanged on Google's real OR-Tools on a computer or in Colab.**

◯ True ◯ False

✎ **If you could ask a solver to plan one thing at your school, what would it be? Write one rule it must keep, and one rule about fairness.**

Check your answers

Tell the computer

Optimization · Level 2 · Lesson 4

1 No plan keeps all the rules

INFEASIBLE means the solver searched and proved that no plan keeps every rule, for example asking for 2 exam slots when three of one pupil's exams all clash.

2 Top to bottom: 4, 5, 1, X, 3, 2, 8, X, 6, 7

```
from ortools.sat.python import cp_model
model = cp_model.CpModel()
y = model.new_int_var(0, 10, "beans")
model.add(3 * y <= 18)
model.maximize(40 * y)
solver = cp_model.CpSolver()
solver.solve(model)
print(solver.value(y))
```

Each line needs what came before: the variable needs the model, the limit and goal need the variable, and you can only read the value after solving.

3 True 290

3 + 5 = 8, and 8 is at most 8, so the comparison is True.

4

```
x = model.new_int_var(0, 10, "tomato")
y = model.new_int_var(0, 10, "beans")
model.add(x + y <= 8)
model.add(2 * x + y <= 14)
model.add(x + 3 * y <= 18)
model.maximize(30 * x + 40 * y)
```

new_int_var makes a whole-number variable, like the y line below it.

5 True

Darsi's small OR-Tools is made to match Google's, so your phone and a computer run the same program.